

Reviewer Pack — SoS Cone-Gram Stable Rank Caps AC^0 Depth-d Size for PARITY

Entry 63e0e3f43696 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 00:39:13 UTC

SoS Cone-Gram Stable Rank Caps AC^0 Depth-d Size for PARITY

Entry ID: 63e0e3f43696 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 00:39:13 UTC

1. Conjecture

Field A (mathematical branch): Sum-of-Squares hierarchy / Lasserre stable-rank geometry of structural Gram matrices on cone-indicator features (Lasserre 2001; Parrilo 2003; Barak-Steurer 2014; Schur 1923 Hadamard-product PSD theory; Rudelson-Vershynin 2007 stable rank). The invariant $\psi(C) := \sum_i \lambda_i(K(C)) / \lambda_{\max}(K(C))$ — the Schatten-1/Schatten- ∞ stable rank of the gate-cone-indicator Gram matrix $K(C)[g,h] = |\text{cone}(g) \cap \text{cone}(h)|/n$ — is the SoS-degree-2 second-moment / pseudoexpectation functional of the structural feature ensemble $\{1_{\{\text{cone}(g)\}}/\sqrt{n}\}_{g \in V(C)}$ viewed as a Lasserre relaxation of the gate-incidence polytope. It is a non-ring, real-spectral Schatten-ratio (no F_q polynomial extension preserves $\sum \lambda / \lambda_{\max}$ under polynomial ring lifts), Aaronson-Wigderson algebrization-safe. STRUCTURAL on the circuit DAG (two AC^0 circuits with the same Boolean function can have very different cone Gram spectra), so it sidesteps the Razborov-Rudich barrier that killed the prior ‘Lasserre Moment-Rank Gap of Accept Set’ attempt (which operated on truth-table accept sets). Sym(n)-equivariant under input permutations ($K(C)$ transforms by simultaneous permutation, preserving spectrum), fulfilling the focus’s natural-group-action requirement. An arXiv search ‘Lasserre stable rank’ AND (‘ AC^0 ’ OR ‘PARITY lower bound’ OR ‘switching lemma’) returns 0 direct hits with <5 adjacent papers (all on SoS-Max-Cut / SoS-CSP integrality gaps, never on gate-cone Gram spectra of AC^0 circuits). **Field B** (complexity object): AC^0 circuit size $s(C)$ and depth d for the explicit n -bit PARITY function, in the Håstad 1986 switching-lemma / Razborov-Smolensky / Beame 1994 regime, accessed through the depth- d $2^{\Theta(n^{1/(d-1)})}$ lower bound (open: tight constants and a non-restriction-based proof). In the Cook-Reckhow-Krajíček bounded-arithmetic framework, an AC^0 PARITY size lower bound is the canonical $V^0 \not\sqsubseteq V^0(\oplus)$ separation stepping stone.

Statement:

For every AC^0 circuit C of depth $d \geq 2$ on n input variables that computes PARITY $_n$ exactly, the gate-cone-indicator Gram matrix $K(C) \in \mathbb{R}^{|V| \times |V|}$ with $K[g,h] := |\text{cone}(g) \cap \text{cone}(h)|/n$ satisfies $\psi(C) := (\sum_i \lambda_i(K(C))) / \lambda_{\max}(K(C)) \leq C_0 \cdot (\log_2 s(C))^{d-1}$, equivalently $\log_2 s(C) \geq \psi(C)^{1/(d-1)/C_0}$, for an absolute constant $C_0 \leq 4$. The bound is non-vacuous because PARITY requires every input in $\text{cone}(\text{output})$, forcing $\psi(C)$ to scale at least as $n^{(d-2)/(d-1)}$ on any depth- d AC^0 realization, matching Håstad’s lower-bound shape without invoking random restrictions. A single AC^0 circuit C computing PARITY with $\psi(C)^{1/(d-1)} > 4 \cdot \log_2 s(C)$ falsifies the conjecture.

Rationale (proposer’s reasoning):

The cone Gram matrix $K(C)$ is the natural SoS-degree-2 Lasserre moment-matrix of the structural feature ensemble $\{1_{\{\text{cone}(g)\}}\}_g$, and its stable rank $\psi(C)$ is a Schur-Horn-

type combinatorial second-moment measuring how many ‘effective independent input directions’ the wiring of C must explore — exactly the resource that Håstad’s switching lemma shows PARITY forces large via random restrictions, but here computed deterministically from the DAG alone. Because $\text{cone}(g)$ depends only on the wiring (not the gate functions or truth table), two AC^0 circuits with identical Boolean function can have very different $\psi(C)$, giving a Razborov–Rudich-safe structural functional within the SoS hierarchy. The bound’s exponent $(d-1)$ mirrors Håstad’s tight $2^{\Theta(n \{1/(d-1)\})}$ shape, so if it holds even loosely it yields a restriction-free route to AC^0 PARITY lower bounds and a candidate ψ for the focus’s polynomial-time natural-invariant target.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): be86a764f1f3c1f1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 540 AC^0 PARITY instances ($n \in \{4, 6, 8, 10, 12, 14, 16, 20, 24\}$, $d \in \{2, 3\}$, 30 seeds each), compute $r(C) = \psi(C)^{1/(d-1)} / \log_2 s(C)$ where $\psi = \text{trace}(K) / \lambda_{\max}$. SUPPORT iff median $r \leq 4$ AND max $r \leq 8$; FALSIFY iff any seed yields $r > 8$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.70	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Lasserre stable rank AC^0 PARITY lower bound - sum of squares Gram matrix circuit complexity switching lemma - cone indicator spectral stable rank bounded depth circuit

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def reverse_bfs(g, start):
    n = len(g)
    visited = [False] * n
```

```

queue = [start]
visited[start] = True
cone = set()
while queue:
    node = queue.pop(0)
    cone.add(node)
    for neighbor in g[node]:
        if not visited[neighbor]:
            visited[neighbor] = True
            queue.append(neighbor)
return cone

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find the pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

    # Back-substitute to find the solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = A[i][-1]
        for j in range(i+1, n):
            x[i] -= A[i][j] * x[j]
        x[i] /= A[i][i]

    return x

def power_method(A, max_iter=30):
    n = len(A)
    v = [random.random() for _ in range(n)]
    v = [x / sum(v) for x in v] # Normalize

    for _ in range(max_iter):
        Av = [sum(A[i][j] * v[j] for j in range(n)) for i in range(n)]
        lambda_max = max(abs(x) for x in Av)
        v = [x / lambda_max for x in Av]

    return lambda_max, v

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [4, 6, 8, 10, 12, 14, 16, 20, 24]
    d_values = [2, 3]
    results = []

```

```

for n in n_values:
    for d in d_values:
        for _ in range(30):
            # Generate a random permutation of input variables
            inputs = list(range(n))
            random.shuffle(inputs)

            # Construct the circuit
            g = [[] for _ in range(n)]
            if d == 2:
                for i in range(n):
                    g[i].append((i, 'AND'))
                    g[i].append((i, 'OR'))
            elif d == 3:
                block_size = math.ceil(math.sqrt(n))
                for i in range(block_size):
                    for j in range(i*block_size, min((i+1)*block_size, n)):
                        g[j].append((j, 'AND'))
                        g[j].append((j, 'OR'))

            # Add random NOT-NOT redundancy gates
            for _ in range(5):
                i = random.randint(0, n-1)
                j = random.choice(g[i])
                if j[1] == 'AND':
                    g[i].append((i, 'NOT', j))
                elif j[1] == 'OR':
                    g[i].append((i, 'NOT', j))

            # Compute the cone of each gate
            cones = [reverse_bfs(g, i) for i in range(n)]

            # Form the Gram matrix K(C)
            K = [[0] * n for _ in range(n)]
            for i in range(n):
                for j in range(i+1, n):
                    intersection_size = len(cones[i].intersection(cones[j]))
                    K[i][j] = K[j][i] = intersection_size / n

            # Compute the top spectrum of K(C)
            lambda_max, _ = power_method(K)

            # Compute  $\psi(C)$  and  $r(C)$ 
            psi_C = sum(K[i][i] for i in range(n)) / lambda_max
            r_C = psi_C ** (1/(d-1)) / math.log2(2**n)

            results.append(r_C)

mean_r = sum(results) / len(results)
max_r = max(results)

if max_r > 8:
    return {
        "metric_name": "r(C)",
        "metric_value": mean_r,
    }

```

```

        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": f"max r(C) = {max_r} (exceeds threshold)"
    }
else:
    return {
        "metric_name": "r(C)",
        "metric_value": mean_r,
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial["metric_value"])

    mean_r = sum(results) / len(results)
    max_r = max(results)
    support_fraction = sum(1 for r in results if r <= 8) / len(results)

    if max_r > 8:
        print(f"RESULT: FALSIFIED counterexample=\"max r(C) = {max_r}\" first_failing_seed={seeds[0]}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_r} std=0 support_fraction={support_fraction}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c7fd2fe.py", line 151, in <module>
 trial = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c7fd2fe.py", line 108, in run_trial
 cones = [reverse_bfs(g, i) for i in range(n)]

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c7fd2fe.py", line 28, in reverse_bfs
 if not visited[neighbor]:

~~~~~^~~~~~

TypeError: list indices must be integers or slices, not tuple

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

**9. Final verdict & safety rail**

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed with a `TypeError` in `reverse_bfs` (list indexed by a tuple) before producing any `r(C)` values, so neither the support nor falsification condition could be evaluated. | next: Fix the cone-construction bug in `reverse_bfs` (ensure neighbor is an integer node id, not a (node, edge) tuple) and rerun the 540-instance sweep.

**11. Audit log (LLM calls)**

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max    | opus             | 0         | 0   | 188858       |
| 2 | preregistration | claude_max    | opus             | 0         | 0   | 5463         |
| 3 | novelty         | claude_max    | opus             | 0         | 0   | 3216         |
| 4 | novelty         | claude_max    | opus             | 0         | 0   | 8346         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16271        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17774        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14857        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15702        |
| 9 | judge           | claude_max    | opus             | 0         | 0   | 3934         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 274421 ms total latency. Provider mix: {'claude\_max': 5, 'ollama\_remote': 4}

(full prompt+response transcripts available in `research/audit/63e0e3f43696.jsonl`)

**12. Reproducibility**

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/63e0e3f43696.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/63e0e3f43696.tar.gz` (if generated)